

Пространство на състоянията е съвкупността от всички състояния, които са достижими от дадено начално състояние. То може да се представи като граф или дърво, където всяко състояние е връх, а successor функцията е ребро. Когато е представено като дърво началното състояние е коренът на дървото, а крайните състояния и нашето целево състояние (goal state) са представени като дърво.

Стратегиите за търсене в пространството на състоянията се оценяват по:

- пълнота - ако съществува решение винаги ли алгоритъмът го намира
- оптималност - винаги ли алгоритъмът намира най-доброто (оптималното) решение
- времева сложност - как расте броят обхождени върхове
- пространствена сложност - как расте максималният брой върхове, които се пазят в паметта

Времева сложност измерваме пряко:

- b - максимално разклонение на дървото
- d - дълбочината на най-евтиното решение
- m - максималната дълбочина на пространството (може да е ∞)

Глобално срещу локално търсене

- Глобално търсещите алгоритми търсят в цялото пространство на състояния
- Локално търсещите алгоритми търсят само в локална зона от пространството от състояния, а ако решението, което търсят е извън тази зона, то алгоритъмът няма да го открие

Информираниите стратегии за търсене имат информация за целевото състояние, което ги прави по-ефективни. Тази информация възниква от евристична функция, която оценява колко близо си до целевото състояние

Best-first search

За всеки връх се използва евристичната функция за да се сметне неговата желаност и винаги разширяваме върха с най-висока желаност.

Имплементира се с приоритетна опашка сортирана в низходящ ред спрямо евристичната оценка, т.е. $g(n)$ е най-желаният връх.

- пълнота - не (може да влезе в цикъл или безкраен път)
- оптималност - не (игнорира цената до върха и се води само по евристичната)
- време - $O(b^m)$, но добра евристика може да го подобри
- памет - $O(b^m)$, пазят се всички достигнати състояния

Локално търсене в лъч (Beam Search)

Алгоритъм Best-first search с ограничен размер на опашката, т.е. при избирането на следващия възел се вземат само най-добрите l възела

- пълнота - не (същата слабост като Best-first)
- оптималност - не (същата слабост като Best-first)
- време - $O(bl^m)$ - ни са нужни $b \cdot l$ операции и m -ият нива - $b \cdot l^m$
- памет - $O(bl)$

Изкачване на хълмове (Hill Climbing)

Работи подобно на Beam Search, но:

1. опашката е с ограничение $l=1$, т.е. пазим само най-добрия (според h) и също така
 2. избира най-добрия съсед също ако евристичната му оценка е по-добра от тази на родителя
- пълнота - не - спира при локален екстремум
 - оптимален - не - спира при локален екстремум
 - време - $O(bm)$ зависи от дълбочината
 - памет - $O(b)$ обработва всички деца и избира само най-добрия

Симулирано втвърдяване

Подобрене на Hill Climbing, което избягва засядане в локален максимум за евристичната оценка, като допуска лоши ходове, така може да слезе от локален максимум и да стигне до по-добър локален максимум дори и понякога глобален. Протяг лоши ходове намалява с времето, затова се казва и simulated annealing

- пълнен - не (спираме в локален максимум, който по никакъв)
- оптимален - не (низи не гарантира, че е глобален максимум)
- време - зависи от „охлаждането“ (допускането на грешки)
- памет - $O(1)$